

Timetable planner

Relazione di progetto

A cura di:

Brando Calderara

Diego Gaibotti

Indice

Timetable planner.....	.1
Introduzione e scopo del progetto.....	.2
Astrazione / Modellazione del problema reale in PDDL.....	.2
Configurazione dell'ambiente di sviluppo.....	.2
Sviluppo incrementale in versioni.....	.3
La Graphical User Interface con Streamlit.....	.4
Testing e analisi delle soluzioni.....	.4
Grafici e performance.....	.5
Conclusioni.....	.6
Suddivisione del lavoro e contributo.....	.7
Utilizzo di strumenti di intelligenza artificiale.....	.7

Introduzione e scopo del progetto

Il problema della generazione degli orari accademici consiste nell'assegnare in modo coerente un insieme di risorse (docenti, aule e gruppi di studenti) a specifiche fasce orarie, rispettando rigidi vincoli logistici e preferenze organizzative. Tradizionalmente, questo dominio viene affrontato con successo utilizzando dei solver basati sul soddisfacimento di vincoli (CSP). In questo progetto, tuttavia, abbiamo scelto di esplorare una strada alternativa, affrontando la sfida attraverso l'Automated Planning, servendoci del linguaggio PDDL e del risolutore ENHSP.

La motivazione alla base di questa scelta è prettamente didattica e di ricerca: lo scopo principale del progetto non è competere con i software commerciali di scheduling, bensì indagare i limiti strutturali del planning in un dominio per il quale non è stato nativamente concepito.

Attraverso lo sviluppo in modo iterativo di un'applicazione testata su 5 istanze di difficoltà crescente, vogliamo dimostrare empiricamente per quale motivo il planning fatichi a gestire questi scenari. L'obiettivo è identificare il "collo di bottiglia" del sistema, osservando come la crescita delle variabili porti all'esaurimento delle risorse hardware (RAM e CPU) e all'impossibilità di trovare una soluzione in tempi compatibili con le necessità organizzative.

Astrazione / Modellazione del problema reale in PDDL

Per trasformare il *Timetable problem* in linee di codice abbiamo fatto delle semplificazioni, che ci hanno permesso di ottenere dei risultati in tempi molto brevi. L'obiettivo infatti non prevedeva di trovare l'algoritmo perfetto, ma di capire il problema che si manifesta a monte.

Per gestire la settimana abbiamo creato, per ogni giorno, degli oggetti di tipo fascia oraria, ai quali verranno assegnati i corsi. Ogni corso è tenuto da uno ed un solo professore e solo un "gruppoStudenti", ad esempio il gruppo di ITID primo anno, potrà frequentarlo. Solo un docente e solo un gruppo di studenti potranno essere presenti in un'aula e non ci saranno lezioni non assegnate (no alle lezioni facoltative).

Le lezioni possono essere assegnate a gruppi di 2, 3 o 4 ore, prediligendo quest'ultima opzione attraverso un sistema di costi da ottimizzare. Il planner, grazie al predicato "oraCorrente" scandirà in ordine ciascuna fascia oraria e assegnerà la lezione nel modo più conveniente.

A livello logico, il sistema si basa su due tipologie di regole: *hard constraints* e *soft constraints*. I primi costituiscono i vincoli invalicabili, alla base della validità del piano; ad esempio garantiscono che un gruppo di studenti non abbia mai due lezioni sovrapposte e che tutte le lezioni vengano assegnate. I secondi invece rendono l'orario qualitativamente migliore: attraverso la funzione di costo verranno infatti penalizzate le ore buche e l'uso di aule aggiuntive rispetto a quelle strettamente necessarie.

Configurazione dell'ambiente di sviluppo

Lo sviluppo del progetto ha richiesto l'allestimento di un ambiente di lavoro strutturato, capace di far comunicare il motore, ovvero il planner, con l'interfaccia scritta in Python.

Per descrivere il *problem* e il *domain* abbiamo utilizzato il linguaggio PDDL (Planning Domain Definition Language), lo standard per scrivere problemi di planning. Come motore invece abbiamo adottato ENHSP (Expressive Numeric Heuristic Search Planner), un planner che supporta le espressioni numeriche, essenziale per la gestione dei soft constraints secondo le logiche usate nel progetto. Inoltre, abbiamo testato il planner Patty, scritto in Python, in quanto "numeric planner", ma con scarsi risultati e performance.

Per garantire una corretta esecuzione del planner abbiamo usato un file Batch (.bat). Configurato per interfacciarsi con l'estensione PDDL, questo file si occupa di passare i file di dominio e problema direttamente all'eseguibile Java di ENHSP, registrandone l'output per l'analisi e permettendo l'impostazione delle flag. Abbiamo usato prevalentemente le due seguenti configurazioni:

```
java -jar "%~dp0.\enhsp25.jar" -o %1 -f %2 -planner sat-hadd -sjr -sp "%~dp0logsPDDL\saved_plan_output_%TIMESTAMP%.txt"
```

per la ricerca di una soluzioni che soddisfi gli hard constraints in breve tempo;

```
java -jar "%~dp0.\enhsp25.jar" -o %1 -f %2 -planner opt-hrmax -timeout 360
```

per trovare la miglior soluzione in assoluto, la soluzione ottimale. Essendo un'operazione computazionalmente onerosa abbiamo inserito la flag “-timeout 360”, che interrompe forzatamente la ricerca dopo 6 minuti di esecuzione per prevenire l'esaurimento delle risorse.

La flag “-anytime” è stata utile nel testing per valutare la performance anche quando l'esecuzione era lenta.

Per analizzare i tempi di esecuzioni del sistema è importante conoscere la distinzione tra due fasi dell'esecuzione di ENHSP: il grounding e il planning. Il grounding è la fase preparatoria in cui il planner traduce le azioni generiche scritte nel file dominio in azioni specifiche, istanziando tutte le possibili combinazioni con gli oggetti definiti nel file problema. Questa fase è molto dispendiosa in termini di risorse ed è il “collo di bottiglia” del processo risolutivo. Il planning invece è la fase di ricerca vera e propria, in cui l'algoritmo naviga lo spazio degli stati (generato dal grounding) per trovare il percorso che conduce al goal.

Sviluppo incrementale in versioni

Per la stesura del codice PDDL abbiamo seguito un approccio incrementale e parallelo. Scrivere due diverse implementazioni di domain e problem ci ha permesso infatti di testare l'efficacia delle due diverse logiche. Ciascuna implementazione ci ha permesso di introdurre complessità, constraints e variabili. La versione finale (V5) di una delle implementazioni parallele ha dimostrato di essere più efficiente, di seguito la descrizione delle iterazioni che ne sono alla base.

La *versione 1* si concentra sulla logica alla base del problema dello scheduling, ovvero l'assegnazione dei corsi alle fasce orarie. La gestione dello scorrere del tempo è stata la difficoltà principale riscontrata, risolta attraverso l'impiego di una struttura simile ad una linked list, ovvero una concatenazione delle fasce orarie. La lista veniva letta grazie al predicato “oraCorrente” e all'azione “scorriTempo”. Tuttavia, i primi test hanno restituito tempi di risoluzione piuttosto lunghi. Il sistema faticava a muoversi agilmente poiché mancavano delle logiche di interruzione anticipata come la pausa pranzo o il salto al giorno successivo.

La *versione 2* parte dalle criticità della V1 e introduce quanto sopracitato: l'azione “pranzo” e l'azione “terminaGiornata”. In questa fase è entrata in gioco l'ottimizzazione tramite soft constraints. Abbiamo manipolato la funzione di costo per guidare il planner verso orari qualitativamente migliori: all'azione base “scorriTempo” è stato assegnato un costo maggiore rispetto alle due nuove azioni. Questa scelta ha disincentivato intrinsecamente la creazione di “ore buche”. Il risultato è stato eccellente: il planner è diventato incredibilmente efficiente, riuscendo a trovare una soluzione esplorando appena 240 stati.

La *versione 3* eredita la stabilità della V2, è stata quindi teatro di aggiunte e cambiamenti. Il problem è stato esteso, con l'aggiunta di aule e docenti. Abbiamo strutturato la logica immaginando un orario indipendente per ogni singola aula. L'introduzione di queste nuove variabili ha richiesto un'attenta calibrazione degli hard constraints per prevenire qualsiasi tipo di sovrapposizione. Come previsto a livello teorico, l'aumento degli object ha iniziato ad avere le prime ricadute visibili sui tempi di grounding.

La *versione 4* ha completato il modello introducendo i “gruppiStudenti”, un oggetto che indica i (dall’inglese) *cohorts*, come ad esempio il terzo anno ITID o il primo di ingegneria informatica. In questo modo è stata resa possibile la generazione di orari per più corsi di studio contemporaneamente. In questa fase è emersa la struttura dell’output finale: un orario per ogni gruppoStudente che possa essere poi convertito in immagine o formato tabellare. Sottoponendo la V4 a cicli di test più ampi, abbiamo iniziato a scontrarci concretamente con i limiti architetturali del planning applicato allo scheduling, registrando un grande aumento degli stati espansi.

La versione 5 integra, alla V4, del codice scritto in Python che funge da problem generator. Questa automazione ha permesso di gestire l’input di dati da un file *xlsx*, modificabile dai più noti e familiari al pubblico gestori di fogli di calcolo. La scrittura di questo script Python per ciascuna delle V4 parallele che leggeva il medesimo *xlsx* ha permesso un confronto rapido e diretto delle due implementazioni. Il problem generator ci ha permesso di rendere subito accessibile all’utente il nostro applicativo, richiedendo solo la capacità di scrivere fogli di calcolo.

Il problem generator sfrutta la libreria *Pandas* di Python per leggere il contenuto dei file *xlsx* e ne traduce il contenuto in variabili e strutture. Ad esempio, le “disponibilità” di docenti, aule e gruppiStudenti vengono lette dall’*xlsx* convertendo 0 e 1, che rispettivamente significano, occupato/indisponibile e libero. Abbiamo sfruttato in questo modo la Closed World Assumption, in modo da non dover scrivere il valore di verità dei predicati negativi (non è necessario specificare che un professore è disponibile, ovvero la condizione normalmente più frequente).

La Graphical User Interface con Streamlit

Per rendere il sistema fruibile, nascondere la complessità del codice e per armonizzare i diversi elementi, abbiamo sviluppato un’interfaccia grafica utilizzando Streamlit, una libreria Python open source per la creazione di web app. La GUI è strutturata in un sistema a schede e gestisce l'intero flusso di lavoro attraverso tre fasi principali: la gestione degli input, la fase di running e la fase di restituzione dell’output.

La prima fase prevede il caricamento di un file *xlsx* (o l’utilizzo della configurazione precedente). Il file, grazie a *Pandas*, viene reso leggibile e modificabile in loco dall’utente. La tab “Utilizzo” permette poi di compiere il processo che separa input e output. Lo fa guidando l’utente passo passo, grazie a delle flag di controllo.

Dopo aver salvato le modifiche con il pulsante “Save to storage” all’utente verrà data la possibilità di mandare in esecuzione il planner, tramite apposito pulsante. I dati principali riguardanti i tempi di esecuzione verranno mostrati in primo piano e, per i più curiosi, dei dati aggiuntivi verranno mostrati al di sotto.

La scheda “Output” permetterà all’utente di visualizzare in formato tabellare il risultato, ottenuto tramite la lettura del plan generato da ENHSP. L'algoritmo analizza il file riga per riga, filtrando solo le azioni che iniziano con il prefisso “assegna”. Analizzandole l'algoritmo capisce quante ore consecutive deve occupare. Infine, questi dati vengono convertiti in DataFrame *Pandas* e mostrati a schermo, restituendo all'utente un orario settimanale pulito, suddiviso in tabelle distinte per ogni singolo gruppo di studenti.

Testing e analisi delle soluzioni

L'obiettivo principale di questo progetto non era la mera stesura del codice, ma l'analisi critica del comportamento di un sistema di Automated Planning applicato alla generazione di Timetable per orari universitari. Per dimostrare empiricamente i limiti di questo approccio, abbiamo sottoposto il sistema a dei test.

L'osservazione dei tempi di calcolo durante lo sviluppo incrementale ci ha fornito i primi indizi sul comportamento del planner: nelle prime fasi (V1), il sistema impiegava molto tempo, in relazione al numero di variabili, per la mancanza di vincoli stringenti. Con l'introduzione (nella V2) dei soft constraints e l'ottimizzazione dei costi, l'algoritmo è diventato molto più efficiente e ci consentiva di trovare una soluzione esplorando appena 240 stati. Il vero punto di svolta in negativo è avvenuto con la V3 e la V4. L'introduzione di nuovi oggetti ha innescato un'immediata crescita dei tempi di esecuzione. Il branching factor è aumentato. Per misurare empiricamente la crescita dei tempi abbiamo sfruttato quanto già era stato costruito. Abbiamo creato 5 fogli di calcolo, ai quali abbiamo assegnato un livello di difficoltà tramite la modifica del numero di oggetti. I dati sono stati registrati effettuando la media delle misurazioni dei tempi di esecuzione del planner con i diversi problem generati. Per studiare meglio l'impiego di risorse del nostro sistema abbiamo eseguito i test su due macchine diverse: un computer fisso con CPU Intel i5-13500 (13th Gen) equipaggiato con 32GB di memoria RAM e un portatile con una CPU Ryzen 7 (2020) e soli 8GB di RAM, rivelatisi in effetti non sufficienti per gestire la tabella più ricca di elementi, "la Difficulty5". I log hanno rivelato un dato fondamentale: il planning non fallisce durante la ricerca della soluzione, ma durante la fase di grounding. Ad esempio nel test "Difficulty 1" sulla macchina 1 abbiamo circa mezzo secondo di grounding time e meno di 40 millisecondi di planning time.

Il collasso registrato invece mettendo alla prova la macchina 2 è, paradossalmente, ciò che volevamo vedere. Durante il grounding infatti, il planner deve tradurre azioni parametriche (come l'assegnazione di un corso) in azioni proposizionali per ogni possibile combinazione di oggetti. Se indichiamo con C i corsi, D i docenti, A le aule e T le fasce orarie, il numero di azioni istanziate nel grounding cresce in modo moltiplicativo secondo la formula: $\#AZIONI = C * D * A * T$

Aggiungere una singola aula non somma 1 all'equazione, ma moltiplica l'intero spazio delle combinazioni. Questo causa un consumo di RAM insostenibile per un normale calcolatore al crescere, anche minimo, della complessità del problema.

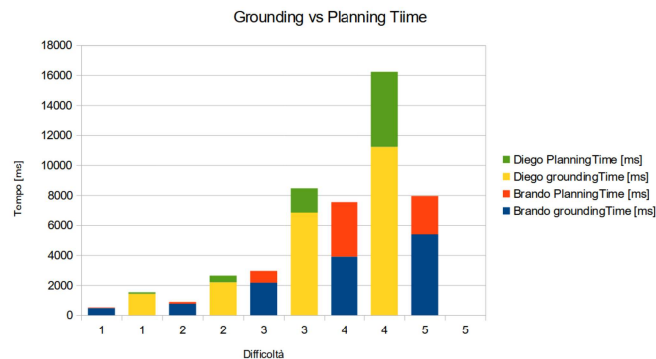
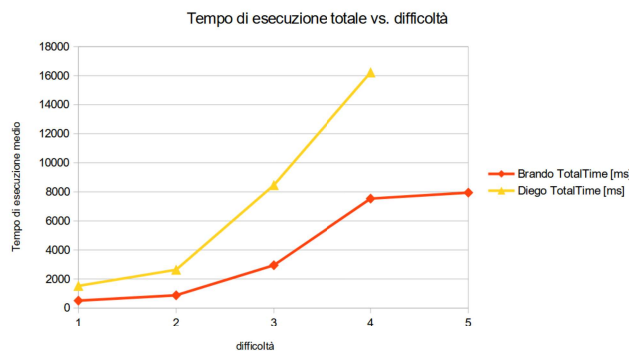
Sebbene i tempi e le risorse limitino pesantemente la scalabilità, dal punto di vista qualitativo il sistema si è rivelato buono. Laddove il planner riesce a completare l'esecuzione sfruttando la flag -planner opt-hrmax, la soluzione trovata è perfetta e coerente coi vincoli imposti considerando le semplificazioni che sono state fatte durante l'astrazione del problema. L'orario generato in output rispetta ovviamente tutti gli hard constraints e minimizza il costo del plan, rendendo l'orario generato teoricamente efficiente per uno studente o, meglio, per l'occupazione delle aule.

Se però ampliamo gli orizzonti e scaliamo leggermente il numero di variabili, il planner risulta inadatto per la creazione di orari perfetti, in quanto il numero di stati da esplorare cresce vertiginosamente.

Grafici e performance

Il grafico a linee evidenzia chiaramente l'andamento delle performance all'aumento delle variabili. Si nota un netto divario tra le prestazioni delle due macchine, mantenendo comunque lo stesso carattere crescente. Si noti che la difficoltà 5 porta l'aggiunta di un singolo corso di studi rispetto alla difficoltà 4, necessario e sufficiente per rendere l'esecuzione impossibile sul pc meno performante.

Questo istogramma ribadisce dove si trova il "bottle neck" del nostro sistema. È evidente come la fase di grounding (parti blu e gialle delle colonne) sia la più dispendiosa, mentre la ricerca della soluzione (parti rosse e verdi delle colonne) generalmente cresce ma in forma meno aggressiva. Anche da questo grafico emerge che il planner fatica molto con la gestione di variabili e vincoli, mentre è molto rapido quando si tratta di navigare alla ricerca di una soluzione.



Conclusioni

Il progetto è nato con l'obiettivo di indagare i limiti teorici dell'Automated Planning, attraverso la risoluzione di un problema normalmente risolto con i CSP.

Alla luce dei test effettuati e dei dati raccolti, possiamo fornire una risposta chiara alla nostra domanda di ricerca iniziale. L'approccio basato sul planning non è la soluzione ideale per problemi di scheduling complessi, nonostante si sia rivelato efficace nella validità e nell'ottimizzazione dell'output (garantendo il rispetto di tutti i vincoli e l'eliminazione delle ore buche) in contesti estremamente ridotti. Il motivo della non scalabilità risiede nella natura del funzionamento del planner.

Il planning nasce per trovare una sequenza di azioni che porti da uno stato iniziale ad un obiettivo, in contrapposizione, lo scheduling non è un problema di raggiungibilità di stati. Un planner dovrà quindi esplorare ogni singola combinazione in modo sequenziale per trovare una soluzione, arrivando alla già citata esplosione dello spazio degli stati.

Suddivisione del lavoro e contributo

Il progetto è stato sviluppato in modo collaborativo, sfruttando lo sviluppo contemporaneo grazie a dei meeting e parallelo tramite Git e [GitHub](https://github.com/DiegoG-web/PDDL_timetableProject) (github.com/DiegoG-web/PDDL_timetableProject). Ogni fase è stata affrontata in modo condiviso ma, in base alle diverse competenze, il contributo di ciascuno è maggiore o minore nelle varie fasi. Di seguito elenco le fasi e chi ha guidato lo sviluppo:

Configurazione iniziale dell'ambiente	Brando Calderara
Sviluppo codice PDDL parallelo	Entrambi
Sviluppo della versione scelta	Diego Gaibotti
Sviluppo del problem generator, logica di base	Brando Calderara
Adattamento del problem generator alla versione scelta	Diego Gaibotti
Sviluppo GUI con Streamlit	Brando Calderara
Stesura e formattazione della <i>Relazione di progetto</i>	Diego Gaibotti

Utilizzo di strumenti di intelligenza artificiale

Supporto allo sviluppo del problem generator: l'AI è stata utilizzata per l'individuazione delle librerie di Python utilizzate per la gestione dei dati.

L'AI è stata utilizzata per comprendere alcuni problemi da linea di comando, a partire dall'installazione dell'ambiente fino al riconoscimento di errori relativi alla versione di Streamlit.

Utilizzo in fase di correzione della relazione.